Unix/Linux Essentials for C Programming

Before formally Starting C Language

Dr. Sandesh Gupta

Understand it. Try it. Build something.

I know what I said in the last class...

- **Last class:**
  - Introduction to the course
  - Basics of computers
- **What I said:**
  "We will start C programming in the next class!"
- **Today's topic:**
  Unix/Linux and terminal commands

Students: "Sir, is C programming running away from us?"

Good news

C programming is not running away.

It is simply waiting for us to prepare its working environment.

Today's promise

Today, we learn where C programs are:

- written,
- compiled, and
- executed.

From the next class, we start C. Seriously!

In one hour, you will control the computer with text

You will be able to:

- explain what Unix/Linux does;
- move safely through its filesystem;
- create, inspect, connect, and protect files;
- compile and run a small C program.

Our destination

```
hello.c
↓ gcc
hello executable
↓
Hello, Unix!
```

2 min

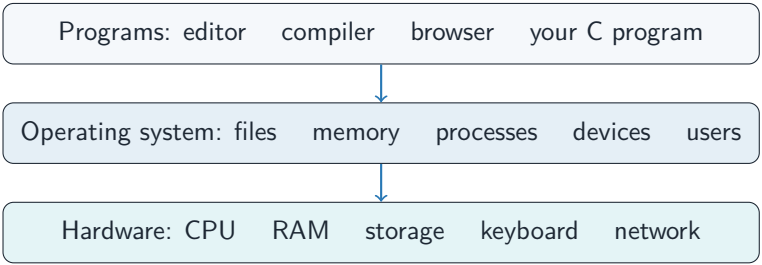
Unix is a family of ideas; Linux is a widely used implementation

- **Unix** began as an operating system and inspired standards and many related systems.
- **Linux** is a Unix-like kernel, usually delivered in a distribution such as Ubuntu or Fedora.
- macOS also provides a Unix command-line environment; commands can differ in details.

For this course

We will say "Unix environment" for the command-line tools and conventions available on the lab's Linux system.

The operating system manages resources for every program



Your program asks the OS for services instead of controlling hardware directly.

Think of the filesystem as an almirah

A physical almirah

- **Almirah** → Filesystem
- **Shelves** → Directories
- **Boxes** → Subdirectories
- **Documents** → Files
- **Labels** → File and directory names

Why use different shelves?

We organize books, certificates, clothes, and bills separately so that we can find them easily.

A student's home directory

```
/home/student/
  |- Documents/
  |      `- marksheet.pdf
  |- Pictures/
  |      `- photo.jpg
  `- c-programs/
       `- hello.c
```

The complete address

Just as we say:

Almirah → Study shelf → C notebook

we write:

`/home/student/c-programs/hello.c`

The Unix filesystem is one directory tree

```
/
|- bin/        Essential commands
|- etc/        System configuration
|- home/       Users' directories
|   `- student/
|        |- Documents/
|        `- c-programs/
|- tmp/        Temporary files
|- usr/        Programs and libraries
`- var/        Logs and changing data
```

The root directory

The directory / is the starting point of the entire filesystem.
Every file and directory exists somewhere below it.

- Directories can contain files and other directories.
- Your personal files are normally stored inside `/home`.
- For example:

`/home/student/c-programs/hello.c`

Paths tell Unix where a file or directory is located

Absolute path

Starts from the filesystem root `/`.

`/home/student/c-programs/hello.c`

It identifies the same location regardless of your current directory.

Relative path

Starts from your current working directory.

`c-programs/hello.c`

Its meaning depends on where you currently are.

Symbol	Meaning
/	Root of the filesystem
~	Your home directory
.	Current directory
..	Parent directory

Kernel, shell, and terminal play different roles

Terminal

The window that sends keystrokes and displays text.

Shell

The program that reads commands, expands them, and starts programs.

Kernel

The core that manages CPU, memory, devices, files, and processes.

You type → shell interprets → kernel provides services → output appears

The prompt tells you the shell is ready

```
student@labpc:~/c-lab$ date
Tue Jul 21 10:30:00 IST 2026
student@labpc:~/c-lab$
```

- Type after `$` ; press `Enter` to run.
- Output appears, then a new prompt returns.
- `Ctrl+C` usually interrupts a command that is still running.

Caution: Do not type the prompt text itself.

Commands follow a predictable grammar

```
command [options] [arguments]
ls      -l        src
gcc     -Wall     hello.c -o hello
```

- **command**: program or shell feature to run
- **options**: modify behavior; often begin with `-` or `--`
- **arguments**: the items to act on
- spaces separate words; spelling and letter case matter
- `Tab` completes names; `Up` recalls a command—edit and reread it

Good habit: Read a command aloud before running it: "list, long format, directory src."

The filesystem is one tree rooted at /

- `/` — root of the whole tree
- `/home/student` — your home directory
- `/etc` — system configuration
- `/tmp` — temporary files
- `/usr/bin` — many commands

A path is an address

Absolute: starts at root
`/home/student/c-lab/hello.c`

Relative: starts here
`c-lab/hello.c`

Three commands answer: where am I, what is here, where next?

```
$ pwd
/home/student
$ ls
c-lab  notes.txt
$ cd c-lab
$ pwd
/home/student/c-lab
```

- `pwd` : print working directory
- `ls` : list directory contents
- `cd PATH` : change working directory

3 min

Special path symbols make navigation faster

```
cd ..       # parent directory
cd .        # current directory (no movement)
cd ~        # your home directory
cd          # also your home directory
cd -        # previous directory
```

Try before moving on

Run `pwd`; move into a directory; run `pwd`; return with `cd -`.

Listing options reveal details and hidden names

```
ls          # names
ls -l       # long listing
ls -a       # include names beginning with .
ls -lh      # long listing, readable sizes
```

- Options can often be combined: `-l` + `-h` → `-lh`.
- Names beginning with a dot are hidden by convention, not secret.

Create a safe workspace before experimenting

```
mkdir unix-practice
cd unix-practice
mkdir src notes
touch notes/commands.txt
ls -R
```

- `mkdir` : make directory
- `touch` : create an empty file, or update its timestamp
- `ls -R` : list recursively

Good habit: Practice inside one clearly named directory.

Copy, move, and rename with explicit destinations

```
cp notes/commands.txt notes/backup.txt
mv notes/backup.txt notes/old-commands.txt
cp -i hello.c hello-copy.c
mv -i hello-copy.c src/
```

- `cp SOURCE DESTINATION` copies.
- `mv SOURCE DESTINATION` moves or renames.
- `-i` asks before overwriting an existing file.

Removal is usually immediate—pause before Enter

```
rm -i unwanted.txt      # remove a file, ask first
rmdir empty-directory  # only if empty
rm -r old-project       # recursive: directory and conten
```

Safety checklist

Run `pwd`. List the target. Check spelling. Prefer `-i`. Never experiment with `rm -r` outside your practice directory.

Caution: The terminal may not provide a recycle bin or undo.

Use simple tools to inspect text files

```
cat hello.c          # print a short file
less long-output.txt # scroll; q quits
head -n 5 data.txt   # first five lines
tail -n 5 data.txt   # last five lines
wc -l hello.c        # count lines
```

- Use `cat` for short content; `less` for longer content.
- In `less`: arrows scroll, `/word` searches, `q` quits.

Search content with grep; locate files with find

```
grep "printf" hello.c
grep -n "TODO" *.c
find . -name "*.c"
```

- `grep PATTERN FILE` : print matching lines.
- `-n` : include line numbers.
- `find START TEST` : walk the directory tree.

Wildcards let one pattern select many names

```
ls *.c        # any name ending in .c
ls test?.c    # exactly one character at ?
ls [ab]*.txt  # starts with a or b, ends in .txt
```

- The **shell expands** wildcards before the command runs.
- First preview matches with `ls PATTERN`; then use the pattern.
- Quote spaces: `mkdir "Week 1"`; prefer simple names such as `week-1`.

Caution: A broad wildcard with `rm` can remove more than intended.

Redirection sends output to a file

```
echo "gcc -Wall hello.c -o hello" > commands.txt
date >> commands.txt
cat commands.txt
```

- `>` creates or overwrites the destination.
- `>>` appends to the destination.
- `<` supplies a file as standard input: `program < input.txt`.

Caution: Inspect important files before using `>`.

A pipe connects one command's output to another's input

```
ls -l | less
grep -n "printf" *.c | less
find . -name "*.c" | wc -l
```

`producer | filter | consumer`

Unix habit

Use small tools that do one job well, then combine them.

Permissions say who may read, write, or execute

```
$ ls -l hello.c hello
-rw-r--r-- 1 student students 92 Jul 21 10:40 hello.c
-rwxr-xr-x 1 student students 15960 Jul 21 10:41 hello
```

Symbol	Meaning	For a file
r	read	view contents
w	write	change contents
x	execute	run as a program

Groups are shown in order: **owner**, **group**, **others**.

Change only the permissions you understand

```
chmod u+x script.sh   # owner: add execute
chmod go-w report.txt # group/others: remove write
```

- Who: `u` user/owner, `g` group, `o` others, `a` all
- Operation: `+` add, `-` remove, `=` set exactly
- Permission: `r`, `w`, `x`

Caution: Do not use `chmod 777` as a universal fix; it grants everyone broad access.

A process is a running instance of a program

```
ps                  # your processes
ps aux | less       # wider system view
top                 # live view; q quits
sleep 30 &          # run in background
jobs                # shell's background jobs
```

- Every process has a process ID (PID).
- `&` returns the prompt while a command runs.
- `Ctrl+C` requests that the foreground process stop.
- `kill PID` requests a graceful stop; `kill -9 PID` is a last resort.

The system already contains its own manual

```
man ls          # manual page; q quits
ls --help       # concise GNU/Linux help
apropos "copy files"
type cd         # what will the shell run?
```

- In `man`: `/pattern` searches; `n` finds the next match.
- Read the **SYNOPSIS** carefully: brackets usually mean optional.

Good habit: Look up options instead of guessing.

Choose an editor you can exit confidently

nano: simplest start

```
nano hello.c
```

`Ctrl+O` save, `Enter` confirm

`Ctrl+X` exit

vim: powerful, modal

```
vim hello.c
```

`i` insert, `Esc` command mode

`:wq` save/quit, `:q!` discard

Graphical editors such as VS Code may be available, but the terminal workflow remains the same.

Editor demonstration: 2 min

Our first C source file gives the compiler instructions

```c
#include <stdio.h>

int main(void) {
        printf("Hello, Unix!\n");
        return 0;
}
```

Save this as `hello.c`. The `.c` extension identifies C source code.

gcc translates source code into an executable

```
gcc -Wall -Wextra -std=c11 hello.c -o hello
```

- `gcc` : compiler driver
- `-Wall -Wextra` : useful warnings
- `-std=c11` : choose a C language standard
- `-o hello` : name the output executable

A successful compile is often silent

Check with `ls -l hello` ; fix warnings even when an executable was produced.

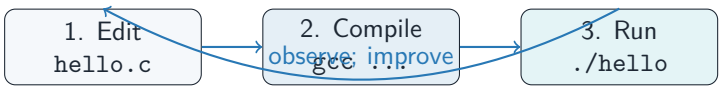
Run a local program with ./ and check its status

```
$ ./hello
Hello, Unix!
$ echo $?
0
```

- `./hello` means "the file named hello in this directory."
- The shell searches command directories, but usually not the current directory.
- Exit status `0` conventionally means success; nonzero means a problem.

Live build and run: 3 min

The edit–compile–run loop is your core workflow



Compiler errors point back to the source. Runtime behavior appears only after a successful build.

Read compiler messages from the first error downward

```
hello.c:4:31: error: expected ';' before 'return'
4 |     printf("Hello, Unix!\n")
  |                               ^
  |                               ;
```

1. Identify `file:line:column`.
2. Read the message and nearby source.
3. Fix the first error; compile again.

Later messages may be consequences of the first mistake.

Most first-day failures have small, fixable causes

Symptom	Likely cause	Check
`command not found`	misspelling or unavailable program	spelling; `type gcc`
`No such file`	wrong directory/name/case	`pwd`; `ls`
`Permission denied`	not executable or access blocked	`ls -l`; correct file
old output remains	source not saved or compile failed	save; read compiler output
program appears stuck	waiting or looping	input; then `Ctrl+C`

Work safely: verify context, target, and effect

1. **Context:** Where am I? `pwd`
2. **Target:** What names match? `ls` or a preview
3. **Effect:** Does this overwrite, recurse, or remove?
4. **Authority:** Avoid `sudo` unless an instructor explicitly requires it.

Never paste an unexplained command

Especially avoid downloads piped directly into a shell. Read and understand commands before running them.

Hands-on challenge 1: build a small directory tree

Goal — 4 minutes

Inside your home directory, create `unix-practice` with subdirectories `src` and `notes`. Create an empty `notes/commands.txt`.

```
unix-practice/
|-- notes/
|   `-- commands.txt
`-- src/
```

Verify using `pwd` and `ls -R`. Pair-check a neighbor's result.

Hands-on challenge 2: create, compile, and run C

1. Save the program as `unix-practice/src/hello.c`.
2. Compile with warnings enabled.
3. Run it and confirm exit status `0`.
4. Redirect its output into `../notes/output.txt`.

```
gcc -Wall -Wextra -std=c11 hello.c -o hello
./hello
./hello > ../notes/output.txt
cat ../notes/output.txt
```

6 min

Quick quiz: choose before the answer appears

1. Which command prints the current directory?
   A. `ls` B. `pwd` C. `cd`
2. What does `..` mean?
   A. home B. root C. parent directory
3. Which operator appends output?
   A. `>` B. `»` C. `|`

Answers

1: B 2: C 3: B

Quick quiz: reason about commands

1. What does `grep -n "main" *.c` do?
2. Why is `./hello` used instead of `hello`?
3. What should you check before `rm -r old-project`?

Possible answers

Searches all matching C files for “main” and prints line numbers; explicitly selects the local executable; verify location, target contents, spelling, and whether removal is truly intended.

A small vocabulary now unlocks a complete workflow

- orient: `pwd`, `ls`, `cd`
- manage: `mkdir`, `cp`, `mv`, `rm`
- inspect: `cat`, `less`, `grep`
- combine: `>`, `»`, `|`
- understand: `ls -l`, `ps`, `man`
- build: edit → `gcc` → `./program`

The transferable habit

Check where you are, predict what a command will do, run it, then inspect the result.

Your next step

Repeat the edit–compile–run loop until it feels ordinary.

Practice all these commands in next lab

Compact command reference

Task	Command	Task	Command
location	`pwd`	list	`ls -lah`
move	`cd PATH`	create directory	`mkdir NAME`
create empty file	`touch FILE`	copy	`cp SRC DEST`
move/rename	`mv SRC DEST`	remove file	`rm -i FILE`
view	`less FILE`	search text	`grep -n PAT FILE`
find names	`find . -name "*.c"`	permissions	`ls -l`
processes	`ps` / `top`	help	`man COMMAND`
compile	`gcc -Wall a.c -o a`	run local	`./a`

Caution: Options vary by system. Consult `man` on the lab machine.